

Notas Práticas e Exercícios

Disciplina: Programação Orientada a Objetos

{*Anotações para uso pessoal}

1 Classes e Objetos

Uma classe é uma estrutura que define um tipo de objeto. Uma classe descreve as propriedades e comportamentos dos objetos que serão criados a partir dela. Por exemplo, se você estiver criando um programa de gerenciamento de biblioteca, uma classe Livro poderia conter propriedades como título, autor e número de páginas, bem como métodos para emprestar e devolver o livro.

Um objeto, por sua vez, é uma instância de uma classe. É uma entidade individual que possui características específicas e pode realizar ações conforme definido pela classe. Por exemplo, se você criar um objeto meuLivro a partir da classe Livro, ele poderá ter um título específico, um autor específico e assim por diante.

Em resumo, uma classe é um modelo para criar objetos e define como os objetos devem se comportar e quais propriedades eles devem ter.

2 Exemplos de implementações em JAVA

Implemente esses exemplos e teste! Veja o que acontece em cada linha e teste modificações.

2.1 Exemplo 1

```
1 //classe Pessoa
2 public class Pessoa {
3     // Atributos privados da classe Pessoa
4     private String nome;
5     private int idade;
6
7     // Construtor da classe Pessoa
8     public Pessoa(String nome, int idade) {
9         this.nome = nome;
10        this.idade = idade;
11    }
12
13    // Método para obter o nome da pessoa
14    public String getNome() {
15        return nome;
16    }
17
18    // Método para definir o nome da pessoa
```

```

19     public void setNome(String nome) {
20         this.nome = nome;
21     }
22
23     // Método para obter a idade da pessoa
24     public int getIdade() {
25         return idade;
26     }
27
28     // Método para definir a idade da pessoa
29     public void setIdade(int idade) {
30         this.idade = idade;
31     }
32 }
33
34
35 //outra classe com a main. O código iniciará a execução a partir da main
36 public class MainPessoa {
37     public static void main(String[] args) {
38         // Criando um objeto da classe Pessoa
39         Pessoa pessoa1 = new Pessoa("Ana", 30);
40
41         // Exibindo o nome e a idade da pessoa
42         System.out.println("Nome: " + pessoa1.getNome()); // Saída: Nome:
            Ana
43         System.out.println("Idade: " + pessoa1.getIdade()); // Saída:
            Idade: 30
44
45         // Alterando o nome e a idade da pessoa
46         pessoa1.setNome("Maria");
47         pessoa1.setIdade(25);
48
49         // Exibindo o nome e a idade atualizados da pessoa
50         System.out.println("Nome: " + pessoa1.getNome()); // Saída: Nome:
            Maria
51         System.out.println("Idade: " + pessoa1.getIdade()); // Saída:
            Idade: 25
52     }
53 }

```

2.2 Exemplo 2

```

1 //Classe Circulo
2 public class Circulo {
3     // Atributo privado da classe Circulo
4     private double raio;
5
6     // Construtor da classe Circulo
7     public Circulo(double raio) {
8         this.raio = raio;
9     }

```

```

10
11 // Método para obter o raio do círculo
12 public double getRaio() {
13     return raio;
14 }
15
16 // Método para definir o raio do círculo
17 public void setRaio(double raio) {
18     this.raio = raio;
19 }
20
21 // Método para calcular a área do círculo
22 public double calcularArea() {
23     return Math.PI * raio * raio;
24 }
25 }
26
27 //Outra Classe com o método main
28 public class MainCirculo {
29     public static void main(String[] args) {
30         // Criando um objeto da classe Circulo
31         Circulo circulo1 = new Circulo(5);
32
33         // Exibindo a área do círculo
34         System.out.println("A área do círculo é: " +
35             circulo1.calcularArea()); // Saída: A área do círculo é:
36             78.53981633974483
37
38         // Alterando o raio do círculo
39         circulo1.setRaio(10);
40
41         // Exibindo a área atualizada do círculo
42         System.out.println("A área do círculo é: " +
43             circulo1.calcularArea()); // Saída: A área do círculo é:
44             314.1592653589793
45     }
46 }

```

2.3 Exemplo 3

```

1 //Classe Livro. (arquivo Livro.java)
2 public class Livro {
3     // Atributos privados da classe Livro
4     private String titulo;
5     private String autor;
6     private int anoPublicacao;
7
8     // Construtor da classe Livro
9     public Livro(String titulo, String autor, int anoPublicacao) {
10         this.titulo = titulo;
11         this.autor = autor;

```

```

12         this.anoPublicacao = anoPublicacao;
13     }
14
15     // Método para obter o título do livro
16     public String getTitulo() {
17         return titulo;
18     }
19
20     // Método para definir o título do livro
21     public void setTitulo(String titulo) {
22         this.titulo = titulo;
23     }
24
25     // Método para obter o autor do livro
26     public String getAutor() {
27         return autor;
28     }
29
30     // Método para definir o autor do livro
31     public void setAutor(String autor) {
32         this.autor = autor;
33     }
34
35     // Método para obter o ano de publicação do livro
36     public int getAnoPublicacao() {
37         return anoPublicacao;
38     }
39
40     // Método para definir o ano de publicação do livro
41     public void setAnoPublicacao(int anoPublicacao) {
42         this.anoPublicacao = anoPublicacao;
43     }
44 }
45
46 //Classe com o método main (arquivo MainLivro.java)
47 public class MainLivro {
48     public static void main(String[] args) {
49         // Criando um objeto da classe Livro
50         Livro livro1 = new Livro("O Senhor dos Anéis", "J.R.R. Tolkien",
51             1954);
52
53         // Exibindo os detalhes do livro
54         System.out.println("Título: " + livro1.getTitulo()); // Saída:
55             Título: O Senhor dos Anéis
56         System.out.println("Autor: " + livro1.getAutor()); // Saída: Autor:
57             J.R.R. Tolkien
58         System.out.println("Ano de Publicação: " +
59             livro1.getAnoPublicacao()); // Saída: Ano de Publicação: 1954
60
61         // Alterando o ano de publicação do livro
62         livro1.setAnoPublicacao(1955);

```

```

59
60     // Exibindo o ano de publicação atualizado do livro
61     System.out.println("Ano de Publicação: " +
        livro1.getAnoPublicacao()); // Saída: Ano de Publicação: 1955
62 }
63 }

```

2.4 Exemplo 4

Neste exemplo, a classe `ContaBancaria` simula uma conta bancária com atributos para o nome do cliente e saldo. A classe `MainContaBancaria` demonstra a criação de uma conta, realização de operações como depósito e saque, e também passa o objeto `conta1` como parâmetro para o método `alterarSaldo()`, onde o saldo é modificado.

Observe que, ao passar o objeto `conta1` como parâmetro para o método `alterarSaldo()`, estamos passando apenas a referência para o objeto, não uma cópia do objeto em si. Portanto, qualquer modificação feita dentro do método `alterarSaldo()` afetará o objeto original.

Ao executar o código, você verá que o saldo da conta é alterado tanto dentro do método `alterarSaldo()` quanto na função `main`, demonstrando que apenas a referência para o objeto é passada e que as alterações afetam o objeto original.

```

1 public class ContaBancaria {
2     // Atributos privados da classe ContaBancaria
3     private String nomeCliente;
4     private double saldo;
5
6     // Construtor da classe ContaBancaria
7     public ContaBancaria(String nomeCliente, double saldo) {
8         this.nomeCliente = nomeCliente;
9         this.saldo = saldo;
10    }
11
12    // Método para obter o nome do cliente
13    public String getNomeCliente() {
14        return nomeCliente;
15    }
16
17    // Método para definir o nome do cliente
18    public void setNomeCliente(String nomeCliente) {
19        this.nomeCliente = nomeCliente;
20    }
21
22    // Método para obter o saldo da conta
23    public double getSaldo() {
24        return saldo;
25    }
26
27    // Método para depositar um valor na conta
28    public void depositar(double valor) {
29        saldo += valor;
30        System.out.println("Depósito de " + valor + " realizado. Novo
        saldo: " + saldo);

```

```

31     }
32
33     // Método para sacar um valor da conta
34     public void sacar(double valor) {
35         if (saldo >= valor) {
36             saldo -= valor;
37             System.out.println("Saque de " + valor + " realizado. Novo
38                 saldo: " + saldo);
39         } else {
40             System.out.println("Saldo insuficiente para realizar o saque.");
41         }
42     }
43
44     // Classe com a main (arquivo MainContaBancaria.java)
45     public class MainContaBancaria {
46         public static void main(String[] args) {
47             // Criando um objeto da classe ContaBancaria
48             ContaBancaria conta1 = new ContaBancaria("João", 1000);
49
50             // Realizando algumas operações na conta
51             conta1.depositar(500);
52             conta1.sacar(200);
53
54             // Exibindo o saldo da conta
55             System.out.println("Saldo da conta: " + conta1.getSaldo()); //
56                 Saída: Saldo da conta: 1300
57
58             // Passando o objeto como parâmetro e modificando o saldo
59             alterarSaldo(conta1, 200);
60
61             // Exibindo o saldo atualizado da conta
62             System.out.println("Saldo da conta após alteração: " +
63                 conta1.getSaldo()); // Saída: Saldo da conta após alteração: 1100
64         }
65
66         // Método para alterar o saldo de uma conta
67         public static void alterarSaldo(ContaBancaria conta, double valor) {
68             conta.sacar(valor); // Utilizando o método sacar para modificar o
69                 saldo
70             System.out.println("Saldo alterado na função. Novo saldo: " +
71                 conta.getSaldo());
72             conta = null;
73         }
74     }

```

3 Exercícios Práticos

1. Crie uma classe `Tempo` com três atributos: horas, minutos e segundos. Crie dois construtores: um para inicializar os atributos com valor 0 e outro para inicializar os atributos com valores passados como argumentos. Crie métodos para:
 - a. Funcionar como getter e setter;
 - b. Imprimir os atributos no formato hh:mm:ss;
 - c. Subtrair dois objetos e colocar o resultado no objeto que o chamou;
 - d. Somar que soma dois objetos e colocar o resultado no objeto que o chamou;
 - e. Sobrecarregue este último método para que retorne um objeto com o resultado da operação.
2. Crie uma classe `Estacionamento` para armazenar dados de um estacionamento. Os atributos devem representar a placa e modelo do carro além da hora de entrada e saída do estacionamento. Utilize dois objetos da classe `Tempo` criada no exercício anterior. Crie métodos para:
 - a. Funcionar como getter e setter;
 - b. Inicializar os dados com vazio ou zero;
 - c. Imprimir os dados de um carro estacionado;
 - d. Calcular e retornar o valor a ser pago pelo carro estacionado. Considere o preço de R\$1,50 por hora. Utilize o método da classe `Tempo`.
3. Faça uma classe `Vetor` com um método `soma` que receba dois vetores de inteiros e retorne um terceiro vetor com a soma dos valores dos dois vetores passados como argumento. Sobrecarregue esse método para que também possa receber dois vetores de números decimais e retorne um terceiro vetor com a soma dos valores dos dois vetores passados como argumento.
4. Crie uma classe `Matriz` com um método `soma` que receba duas matrizes de inteiros e retorne uma terceira matriz com a soma dos valores das duas matrizes passadas como argumento. Sobrecarregue esse método para que também possa receber duas matrizes de números decimais e retorne uma terceira matriz com a soma dos valores das duas matrizes passadas como argumento.
5. Crie uma classe `Calculadora` que contenha métodos estáticos para realizar operações básicas de matemática, como soma, subtração, multiplicação e divisão. Certifique-se de tratar casos especiais, como divisão por zero. Teste os métodos da classe sem criar objetos.
6. Desenvolva uma classe `ContadorInstancias` que mantenha o controle do número total de instâncias criadas dessa classe. Implemente um método estático para recuperar o número total de instâncias criadas.
7. Elabore uma classe chamada `StringUtils` com métodos estáticos para manipulação de strings, como converter para maiúsculas, minúsculas, verificar se uma string é um palíndromo, etc. Utilize métodos públicos e privados conforme apropriado para implementar essas funcionalidades.
8. Implemente uma classe `Produto` com atributos como nome, preço e quantidade em estoque. Implemente métodos para adicionar e remover unidades do estoque, além de um método para calcular o valor total do estoque ($\text{preço} \times \text{quantidade}$). Em seguida, crie uma classe `CarrinhoCompras` que represente um carrinho de compras, com métodos para adicionar produtos ao carrinho, calcular o valor total dos itens no carrinho e finalizar a compra, atualizando o estoque de produtos. Implemente uma classe `TesteCompras` com um método `main` para simular o uso do sistema de compras, adicionando produtos ao carrinho, calculando o valor total e finalizando a compra.

9. Programe uma classe `Dado` que represente um dado de seis faces, com métodos para rolar o dado e obter o valor resultante. Em seguida, crie uma classe `JogoDados` com um método `main` para simular um jogo de dados, onde dois jogadores lançam um dado cada um e o jogador com o maior valor vence. O jogo deve rodar até que um dos jogadores alcance uma determinada pontuação máxima.
10. Crie uma classe `JogoTabuleiro` com atributos como nome, ano de lançamento e quantidade em estoque. Implemente métodos para adicionar e remover unidades do estoque, além de um método para verificar se um jogo está disponível. Em seguida, crie uma classe `LojaJogos` com métodos para adicionar jogos ao estoque, pesquisar jogos pelo nome e listar todos os jogos disponíveis. Implemente uma classe `TesteLojaJogos` com um método `main` para testar o sistema de estoque de jogos, adicionando jogos à loja, pesquisando jogos e verificando a disponibilidade.